COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	July 09, 2026	**DATE SUBMITTED:**	
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 1.1 Basic C++ Programming

1. PROCEDURE OUTPUT

ILO A

C++ Program Code:

ILO_A.cpp ILO_B_.cpp

```cpp
#include <iostream>

//Created by Niña Denise Santiago

//Function that takes 2 numbers and displays the sum.
void displaySum(int xNum, int yNum);

//Function that returns whether variable A is greater than variable B in value.
bool greaterThan(int xNum, int yNum);

/*Function that takes (2) boolean values, displays all logical operations
and returns "True" if all operations are successful.*/
bool operations(bool a, bool b);


int main(){
    // Allows display of words true and false instead of 1 or 0
    std::cout << std::boolalpha;

    // Asks for User Input of values
    int num1, num2;
    std::cout << "Enter first number: ";
    std::cin >> num1;
    std::cout << "Enter second number: ";
    std::cin >> num2;

    // Calls Function 1 that display sum
    displaySum(num1, num2);
    std::cout << "-------------------------------------------" << std::endl; // for organization

    // Calls Function 2 that reviews if the value is greater than
    std::cout << "Is " << num1 << " greater than " << num2 << "? " << greaterThan(num1, num2) << std::endl;
    std::cout << "-------------------------------------------" << std::endl; // for organization
```

ILO_A.cpp ILO_B_.cpp

```cpp
    // Asks for Boolean Input
    bool boolA, boolB;
    std::cout << "Enter first boolean[A] (0 for false | 1 for true): ";
    std::cin >> boolA;
    std::cout << "Enter second boolean[B] (0 for false | 1 for true): ";
    std::cin >> boolB;
    std::cout << "-----------------------------------------" << std::endl; // for organization

    // Calls Function 3 that displays logical operations
    bool success = operations(boolA, boolB);
    std::cout << "Logical Operations Execution Success: " << success << std::endl;

    return 0;
}


//void displaySum Definition
void displaySum(int xNum, int yNum) {
    int sum = xNum + yNum;
}

//bool greaterThan Definition
bool greaterThan(int xNum, int yNum){
    return xNum > yNum;
}
```

[*] ILO_A.cpp ILO_B_.cpp

```cpp

//bool operations Definition
bool operations(bool a, bool b){

    //AND Operation
    std::cout << "Operation AND: " << (a && b) << std::endl;
    //OR Operation
    std::cout << "Operation OR: " << (a || b) << std::endl;
    //NOT Operation
    std::cout << "Operation NOT(A): " << (!a) << std::endl;
    std::cout << "Operation NOT(B): " << (!b) << std::endl;
    //XOR Operation
    std::cout << "Operation XOR: " << static_cast<bool>(a ^ b) << std::endl;
    return true;

}
```

Google Colab Link (for Presentation):
https://colab.research.google.com/drive/1Swh3qBwXc6SwHS4njZ19-Xhc1eb1sGOH?usp=sharing

Output

```
Enter first number: 6
Enter second number: 7
-----------------------------------------
Is 6 greater than 7? false
-----------------------------------------
Enter first boolean[A] (0 for false | 1 for true): 0
Enter second boolean[B] (0 for false | 1 for true): 1
-----------------------------------------
Operation AND: false
Operation OR: true
Operation NOT(A): true
Operation NOT(B): false
Operation XOR: true
Logical Operations Execution Success: true
```

<table>
<tr>
<td></td>
<td>

```
Enter first number: 7
Enter second number: 6
-------------------------------------------
Is 7 greater than 6? true
-------------------------------------------
Enter first boolean[A] (0 for false | 1 for true): 1
Enter second boolean[B] (0 for false | 1 for true): 1
-------------------------------------------
Operation AND: true
Operation OR: true
Operation NOT(A): false
Operation NOT(B): false
Operation XOR: false
Logical Operations Execution Success: true
```

```
Enter first number: 6
Enter second number: 6
-------------------------------------------
Is 6 greater than 6? false
-------------------------------------------
Enter first boolean[A] (0 for false | 1 for true): 1
Enter second boolean[B] (0 for false | 1 for true): 0
-------------------------------------------
Operation AND: false
Operation OR: true
Operation NOT(A): false
Operation NOT(B): true
Operation XOR: true
Logical Operations Execution Success: true
```

```
Enter first number: -6
Enter second number: -7
-------------------------------------------
Is -6 greater than -7? true
-------------------------------------------
Enter first boolean[A] (0 for false | 1 for true): 0
Enter second boolean[B] (0 for false | 1 for true): 0
-------------------------------------------
Operation AND: false
Operation OR: false
Operation NOT(A): true
Operation NOT(B): true
Operation XOR: false
Logical Operations Execution Success: true
```

</td>
</tr>
<tr>
<td>In-Depth Code Analysis:</td>
<td>ILO A was a good exercise in learning how to break down problems into smaller functions. I liked how each function had a specific purpose: one for adding numbers, one for comparing values, and one for handling logical operations. It made the program easier to follow and understand. I also found the use of std::boolalpha helpful because it showed true and false instead of just 1 and 0, which made the output more readable. The only thing I noticed was that the displaySum function didn't actually print the sum, so even though it calculates, the</td>
</tr>
</table>

	output never displays the result. Overall, ILO A helped me understand how functions can make code more organized and reusable.

Sections	Answers:

Header File Declaration Section	`#include <iostream>`
	
	`//Created by Niña Denise Santiago`

Global Declaration Section	`//Function that takes 2 numbers and displays the sum.`
	`void displaySum(int xNum, int yNum);`
	
	`//Function that returns whether variable A is greater than variable B in value.`
	`bool greaterThan(int xNum, int yNum);`
	
	`/*Function that takes (2) boolean values, displays all logical operations and returns "True" if all operations are successful.*/`
	`bool operations(bool a, bool b);`

Class Declaration and Method Definition Section	*NOT APPLICABLE*

Main Function	`int main(){`
	`  // Allows display of words true and false instead of 1 or 0`
	`  std::cout << std::boolalpha;`
	
	`  // Asks for User Input of values`
	`  int num1, num2;`
	`  std::cout << "Enter first number: ";`
	`  std::cin >> num1;`
	`  std::cout << "Enter second number: ";`
	`  std::cin >> num2;`
	
	`  // Calls Function 1 that display sum`
	`  displaySum(num1, num2);`
	`  std::cout << "-----------------------------------------" << std::endl; // for organization`
	
	`  // Calls Function 2 that reviews if the value is greater than`
	`  std::cout << "Is " << num1 << " greater than " << num2 << "? " << greaterThan(num1, num2) << std::endl;`
	`  std::cout << "-----------------------------------------" << std::endl; // for organization`
	
	`  // Asks for Boolean Input`
	`  bool boolA, boolB;`
	`  std::cout << "Enter first boolean[A] (0 for false | 1 for true): ";`
	`  std::cin >> boolA;`
	`  std::cout << "Enter second boolean[B] (0 for false | 1 for true): ";`
	`  std::cin >> boolB;`
	`  std::cout << "-----------------------------------------" << std::endl; // for organization`
	
	`  // Calls Function 3 that displays logical operations`
	`  bool success = operations(boolA, boolB);`
	`  std::cout << "Logical Operations Execution Success: " << success << std::endl;`
	
	`  return 0;`
	`}`

Method Definition	

```cpp
//void displaySum Definition
void displaySum(int xNum, int yNum) {
    int sum = xNum + yNum;
}

//bool greaterThan Definition
bool greaterThan(int xNum, int yNum){
    return xNum > yNum;
}

//bool operations Definition
bool operations(bool a, bool b){

    //AND Operation
    std::cout << "Operation AND: " << (a && b) << std::endl;
    //OR Operation
    std::cout << "Operation OR: " << (a || b) << std::endl;
    //NOT Operation
    std::cout << "Operation NOT(A): " << (!a) << std::endl;
    std::cout << "Operation NOT(B): " << (!b) << std::endl;
    //XOR Operation
    std::cout << "Operation XOR: " << static_cast<bool>(a ^ b) << std::endl;
    return true;

}
```

Table 1-0: C++ Structure Code for Answer

COMPUTER ENGINEERING

QUEZON CITY

ILO B	C++ Program Code:

[*] ILO_A.cpp [*] ILO_B_.cpp

```cpp
#include <iostream>

// naming convention for class name: PascalCase (starts with capital)

class Triangle{
private:
    double totalAngle, angleA, angleB, angleC;

public:
    // Constructor
    Triangle(double A, double B, double C);
    void setAngles(double A, double B, double C);
    const bool validateTriangle ();
};


int main () {
    // Driver code
    Triangle set1(40, 30, 110);
    if(set1.validateTriangle()){
        std::cout << "The shape is a valid triangle. \n";
    } else {
        std::cout << "The shape is NOT a valid triangle. \n";
    }

    return 0;
}

Triangle::Triangle(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C) {
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle() {
    return (totalAngle <= 180);
}
```

Sections	Answers:

Header File Declaration Section	`#include <iostream>`
	
	`// naming convention for class name: PascalCase (starts with capital)`

Global Declaration Section	*NOT APPLICABLE*

Class Declaration and Method Definition Section	`class Triangle{`
	`private:`
	`  double totalAngle, angleA, angleB, angleC;`
	
	`public:`
	`  // Constructor`
	`  Triangle(double A, double B, double C);`
	`  void setAngles(double A, double B, double C);`
	`  const bool validateTriangle ();`
	`};`

Main Function	`int main () {`
	`  // Driver code`
	`  Triangle set1(40, 30, 110);`
	`  if(set1.validateTriangle()){`
	`  std::cout << "The shape is a valid triangle. \n";`
	`  } else {`
	`  std::cout << "The shape is NOT a valid triangle. \n";`
	`  }`
	
	`  return 0;`
	`}`

Method Definition	`Triangle::Triangle(double A, double B, double C) {`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	`  totalAngle = A + B + C;`
	`}`
	
	`void Triangle::setAngles(double A, double B, double C) {`
	`  angleA = A;`
	`  angleB = B;`
	`  angleC = C;`
	`  totalAngle = A + B + C;`
	`}`
	
	`const bool Triangle::validateTriangle() {`
	`  return (totalAngle <= 180);`
	`}`

Table 1-1: C++ Structure Code for Answer

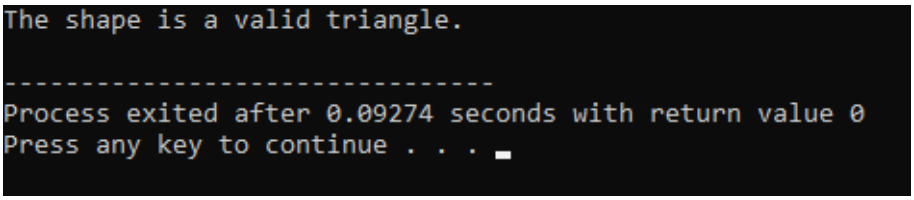
Output	
In-Depth Code Analysis:	In ILO B, I learned how to create a class with private variables and public methods, and how to use a constructor to initialize values. In ILO B, I learned how a constructor works in a class. The constructor Triangle(double A, double B, double C) is called right away when we create a triangle object. It sets the values of the angles and also calculates the total angle, so the object is ready to use without extra steps. This made me see how constructors make code cleaner and save time compared to assigning values one by one. The driver code in main() showed me how to test the class. When we write Triangle set1(40, 30, 110);, the constructor runs automatically and sets up the triangle. Then the driver code calls set1.validateTriangle() to check if the triangle is valid. The program prints the result, which makes it easy to see if the class works correctly. For me, this part connected the idea of “building” an object with the constructor and “using” it in the driver code. It taught me that constructors prepare the object, and the driver code is where we try it out.

Table 1-2: ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. **Create a C++ program to swap the two numbers in different variables.**

C++ Program Code:

```
#include <iostream>
#include <vector>

// Global Declaration
void inputList(std::vector<int> &nums, int size);
void displayList(const std::vector<int> &nums, std::string message);
void swapElements(std::vector<int> &nums, int index1, int index2);

int main() {
    int size;
    std::cout << "Enter how many numbers you want in the list: ";
    std::cin >> size;

    std::vector<int> numbers(size);

    // Input total number of integers in the list
    inputList(numbers, size);

    // Displays the list before swapping
    displayList(numbers, "Before swapping");

    // Asks which positions to swap
    int pos1, pos2;
    std::cout << "\nEnter first position to swap (0-based index): ";
    std::cin >> pos1;
    std::cout << "Enter second position to swap (0-based index): ";
    std::cin >> pos2;

    // Performs swapping
    swapElements(numbers, pos1, pos2);

    // Displays the list after swapping
    displayList(numbers, "After swapping");

    return 0;
}

// Function Definition

// Function to input list of numbers
void inputList(std::vector<int> &nums, int size) {
    std::cout << "\nEnter " << size << " numbers:\n";
    for (int i = 0; i < size; i++) {
        std::cout << "Number " << i << ": ";
        std::cin >> nums[i];
    }
}
```

```cpp
// Function to display list of numbers
void displayList(const std::vector<int> &nums, std::string message) {
    std::cout << "\n" << message << ":" << std::endl;
    for (int i = 0; i < nums.size(); i++) {
        std::cout << nums[i] << " ";
    }
    std::cout << std::endl;
}

// Function to swap two elements in the list
void swapElements(std::vector<int> &nums, int index1, int index2) {
    if (index1 >= 0 && index1 < nums.size() && index2 >= 0 && index2 < nums.size()) {
        int temp = nums[index1];
        nums[index1] = nums[index2];
        nums[index2] = temp;
    } else {
        std::cout << "Invalid indices for swapping!" << std::endl;
    }
}
```

Output:

```
Enter how many numbers you want in the list: 4

Enter 4 numbers:
Number 0: 4
Number 1: 5
Number 2: 6
Number 3: 5

Before swapping:
4 5 6 5

Enter first position to swap (0-based index): 0
Enter second position to swap (0-based index): -1
Invalid indices for swapping!

After swapping:
4 5 6 5
```

```
Enter how many numbers you want in the list: 9

Enter 9 numbers:
Number 0: 1
Number 1: 5
Number 2: 9
Number 3: 3
Number 4: 7
Number 5: 4
Number 6: 6
Number 7: 8
Number 8: 2

Before swapping:
1 5 9 3 7 4 6 8 2

Enter first position to swap (0-based index): 5
Enter second position to swap (0-based index): 7

After swapping:
1 5 9 3 7 8 6 4 2
```

Code Analysis:

Here I utilized vectors to store and manipulate data. I made the program divided into functions for input, display, and swapping, because it makes the code easier to understand, and debug. The error handling for invalid indices was a modification I added since I wanted to prevent mistakes and/or have a crashing program. One thing I realized is that the swapping logic could be simplified by using std::swap, which would make the code shorter and cleaner. However, writing the swap manually helped me understand how values are exchanged between variables. This activity recalled my knowledge in vectors and data handling.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

C++ Program Code:

```cpp
#include <iostream>

// Function declaration
double kelvinToFahrenheit(double kelvin);

int main() {
    double kelvin;

    // Asks for User Input of temperature in Kelvin
    std::cout << "Enter temperature in Kelvin: ";
    std::cin >> kelvin;

    // Calls the conversion computation function
    double fahrenheit = kelvinToFahrenheit(kelvin);

    // Displays result (before and after)
    std::cout << kelvin << " K = " << fahrenheit << " F" << std::endl;

    return 0;
}

// Function definition
double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9.0 / 5.0 + 32.0;
}
```

Output:

```
Enter temperature in Kelvin: 753.91
753.91 K = 897.368 F

--------------------------------
```

Code Analysis:

This program is simple and very straightforward. I created a function that computes the conversions. The function kelvinToFahrenheit(double kelvin) does the conversion using the formula (kelvin - 273.15) * 9.0 / 5.0 + 32.0, and

it returns the result instead of printing it. The separation of printing and computation makes the code cleaner because the function only calculates while main() handles input and output. In main(), the user enters a Kelvin value, the function is called, and the result is displayed in one line. I also used double for variables since temperatures often need decimals. Overall, it's short, clear, and shows how functions can keep calculations organized.

3. Create a C++ program that has a function that will calculate the distance between two points.

C++ Program Code:

```
#include <iostream>
#include <cmath> // for sqrt and pow

// Function declaration
double calculateDistance(double x1, double y1, double x2, double y2);

int main() {
    double x1, y1, x2, y2;

    // Asks for User Input of Point Coordinates
    std::cout << "Enter x1: ";
    std::cin >> x1;
    std::cout << "Enter y1: ";
    std::cin >> y1;
    std::cout << "Enter x2: ";
    std::cin >> x2;
    std::cout << "Enter y2: ";
    std::cin >> y2;

    // Calls function to calculate the Distance
    double distance = calculateDistance(x1, y1, x2, y2);

    // Displays the calculated result
    std::cout << "The distance between (" << x1 << ", " << y1 << ") and ("
              << x2 << ", " << y2 << ") is: " << distance << std::endl;

    return 0;
}

// Function definition
double calculateDistance(double x1, double y1, double x2, double y2) {
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
}
```

Output:

```
Enter x1: 75.21
Enter y1: 12
Enter x2: 67.49
Enter y2: 83
The distance between (75.21, 12) and (67.49, 83) is: 71.4185

--------------------------------
```

Code Analysis:

This program uses the distance formula with sqrt and pow from the cmath library. Similar to the temperature conversion program, I created a function to separate the code/calculation. The function calculateDistance(double x1, double y1, double x2, double y2) is separate from main(), which keeps the code organized. This function simply uses the distance formula. In main(), the program asks for coordinates, calls the function, and prints the result clearly. Using double is good because these coordinates can have decimals. I realized that there is a shorter way or formula for this distance. It would be to use hypot(x2 - x1, y2 - y1), however I proceeded with a more generic or usual approach. Overall, the code is easy to code and recalled my C++ knowledge.

4. Modify the code given in ILO B and add the following functions:
    a. A function to compute for the area of a triangle
    b. A function to compute for the perimeter of a triangle
    c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

C++ Program Code:

```cpp
#include <iostream>
#include <cmath> // for sqrt and pow

class Triangle {
private:
    double sideA, sideB, sideC; // sides of the triangle

public:
    // Constructor
    Triangle(double a, double b, double c);

    void setSides(double a, double b, double c);

    const bool validateTriangle();

    double computeArea();       // Function to compute area
    double computePerimeter();  // Function to compute perimeter
    std::string classifyTriangle(); // Function to classify triangle
};

int main() {
    double a, b, c;

    // User input
    std::cout << "Enter side A: ";
    std::cin >> a;
    std::cout << "Enter side B: ";
    std::cin >> b;
    std::cout << "Enter side C: ";
    std::cin >> c;

    // Creates Triangle object with user-defined sides
    Triangle userTriangle(a, b, c);

    if (userTriangle.validateTriangle()) {
        std::cout << "\nThe shape is a valid triangle.\n";
        std::cout << "Perimeter: " << userTriangle.computePerimeter() << std::endl;
        std::cout << "Area: " << userTriangle.computeArea() << std::endl;
        std::cout << "Classification: " << userTriangle.classifyTriangle() << std::endl;
    } else {
        std::cout << "\nThe shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

```cpp
// Methods definition

Triangle::Triangle(double a, double b, double c) {
    sideA = a;
    sideB = b;
    sideC = c;
}

void Triangle::setSides(double a, double b, double c) {
    sideA = a;
    sideB = b;
    sideC = c;
}

const bool Triangle::validateTriangle() {
    // Triangle inequality theorem
    return (sideA + sideB > sideC &&
            sideA + sideC > sideB &&
            sideB + sideC > sideA);
}

double Triangle::computePerimeter() {
    return sideA + sideB + sideC;
}

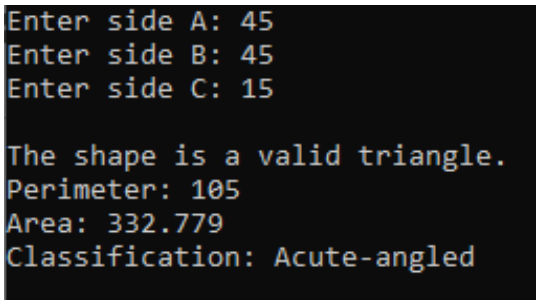
double Triangle::computeArea() {
    double s = computePerimeter() / 2.0; // semi-perimeter
    return std::sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}

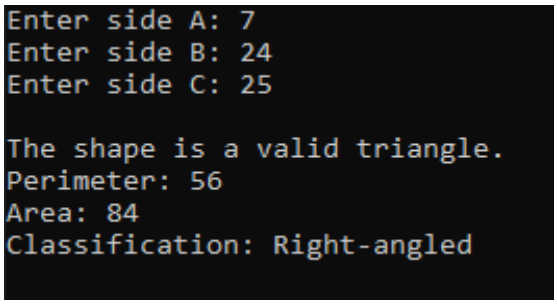
std::string Triangle::classifyTriangle() {
    // Sort sides so that c is the largest
    double a = sideA, b = sideB, c = sideC;
    if (a > b) std::swap(a, b);
    if (b > c) std::swap(b, c);
    if (a > b) std::swap(a, b);

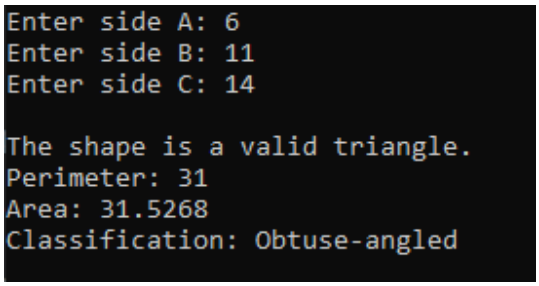
    double lhs = a*a + b*b;
    double rhs = c*c;

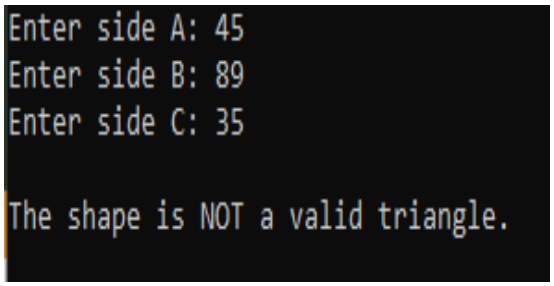

if (std::fabs(lhs - rhs) < 1e-9) {
    return "Right-angled";
} else if (lhs > rhs) {
    return "Acute-angled";
} else if (lhs < rhs) {
    return "Obtuse-angled";
} else {
    return "Others";
}
}
```

Output:









Code Analysis:

This program makes the triangle class more complete by adding methods for perimeter, area, and classification. The constructor sets the sides right away, so the object is ready to use. The validateTriangle() method uses the triangle inequality theorem, which is the correct way to check if the sides form a real triangle. The computePerimeter() method simply adds the sides, while computeArea() uses Heron's formula to calculate the area from side lengths. The classifyTriangle() method compares side lengths to decide if the triangle is right-angled, acute, or obtuse, which adds more functionality. Together, these methods show how a class can handle validation, calculations, and classification all in one organized structure.

Another Version of the Whole Supplementary:

Supplementary.cpp > Triangle

```cpp
#include <iostream>
#include <vector> // for using std::vector (list of numbers)
#include <cmath> // for sqrt and pow
#include <string>
#include <algorithm> // for swap

// ===== No 1: Swap Elements =====
// Function to input a list of numbers
void inputList(std::vector<int> &nums, int size) {
    std::cout << "\nEnter " << size << " numbers:\n";
    for (int i = 0; i < size; i++) {
        std::cout << "Number " << i << ": ";
        std::cin >> nums[i];
    }
}

// Function to display a list of numbers
void displayList(const std::vector<int> &nums, std::string message) {
    std::cout << "\n" << message << ": ";
    for (int num : nums) std::cout << num << " ";
    std::cout << std::endl;
}

// Function to swap two elements in a list
void swapElements(std::vector<int> &nums, int index1, int index2) {
    if (index1 >= 0 && index1 < nums.size() && index2 >= 0 && index2 < nums.size()) {
        std::swap(nums[index1], nums[index2]);
    } else {
        std::cout << "Invalid indices for swapping!" << std::endl;
    }
}

// ===== No 2: Kelvin to Fahrenheit =====
// Function to convert Kelvin to Fahrenheit
double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9.0 / 5.0 + 32.0;
}

// ===== No 3: Distance Between Two Points =====
// Function to calculate distance between two points (x1, y1) and (x2, y2)
double calculateDistance(double x1, double y1, double x2, double y2) {
    return std::sqrt(std::pow(x2 - x1, 2) + std::pow(y2 - y1, 2));
}

```

```cpp
// ===== No 4: Extended Triangle Class =====
// Triangle class with perimeter, area, and classification methods
class Triangle {
private:
    double sideA, sideB, sideC;
public:
    Triangle(double a, double b, double c) {
        sideA = a; sideB = b; sideC = c;
    }

    // Function to validate if the sides can form a triangle
    bool validateTriangle() {
        return (sideA + sideB > sideC &&
                sideA + sideC > sideB &&
                sideB + sideC > sideA);
    }

    // Function to compute the perimeter of the triangle
    double computePerimeter() {
        return sideA + sideB + sideC;
    }

    // Function to compute the area of the triangle using Heron's formula
    double computeArea() {
        double s = computePerimeter() / 2.0;
        return std::sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
    }

    // Function to classify the triangle based on its angles
    std::string classifyTriangle() {
        double a = sideA, b = sideB, c = sideC;
        if (a > b) std::swap(a, b);
        if (b > c) std::swap(b, c);
        if (a > b) std::swap(a, b);

        double lhs = a*a + b*b;
        double rhs = c*c;

        if (fabs(lhs - rhs) < 1e-9) return "Right-angled";
        else if (lhs > rhs) return "Acute-angled";
        else return "Obtuse-angled";
    }
};
```

```cpp
// ===== Main Menu =====
int main() {
    int choice;
    do {
        std::cout << "\n========== Supplementary Activity Menu =========\n";
        std::cout << "1. Swap Elements in a List\n";
        std::cout << "2. Convert Kelvin to Fahrenheit\n";
        std::cout << "3. Calculate Distance Between Two Points\n";
        std::cout << "4. Triangle Calculations (Perimeter, Area, Classification)\n";
        std::cout << "0. Exit\n";
        std::cout << "================================================\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        switch(choice) {
            case 1: {
                int size;
                std::cout << "================================================\n";
                std::cout << "Enter how many numbers you want in the list: ";
                std::cin >> size;
                std::vector<int> numbers(size);
                inputList(numbers, size);
                displayList(numbers, "Before swapping");

                int pos1, pos2;
                std::cout << "Enter first position to swap (0-based index): ";
                std::cin >> pos1;
                std::cout << "Enter second position to swap (0-based index): ";
                std::cin >> pos2;

                swapElements(numbers, pos1, pos2);
                displayList(numbers, "After swapping");
                break;
            }
            case 2: {
                double kelvin;
                std::cout << "================================================\n";
                std::cout << "Enter temperature in Kelvin: ";
                std::cin >> kelvin;
                if (kelvin < 0) {
                    std::cout << "Invalid input! Kelvin cannot be negative.\n";
                } else {
                    double fahrenheit = kelvinToFahrenheit(kelvin);
                    std::cout << kelvin << " K = " << fahrenheit << " F\n";
                }
                break;
            }
```

```
            case 3: {
                double x1, y1, x2, y2;
                std::cout << "=============================================\n";
                std::cout << "Enter x1: "; std::cin >> x1;
                std::cout << "Enter y1: "; std::cin >> y1;
                std::cout << "Enter x2: "; std::cin >> x2;
                std::cout << "Enter y2: "; std::cin >> y2;

                double distance = calculateDistance(x1, y1, x2, y2);
                std::cout << "The distance between (" << x1 << ", " << y1 << ") and ("
                          << x2 << ", " << y2 << ") is: " << distance << std::endl;
                break;
            }
            case 4: {
                double a, b, c;
                std::cout << "=============================================\n";
                std::cout << "Enter side A: "; std::cin >> a;
                std::cout << "Enter side B: "; std::cin >> b;
                std::cout << "Enter side C: "; std::cin >> c;

                Triangle tri(a, b, c);
                if (tri.validateTriangle()) {
                    std::cout << "The shape is a valid triangle.\n";
                    std::cout << "Perimeter: " << tri.computePerimeter() << std::endl;
                    std::cout << "Area: " << tri.computeArea() << std::endl;
                    std::cout << "Classification: " << tri.classifyTriangle() << std::endl;
                } else {
                    std::cout << "The shape is NOT a valid triangle.\n";
                }
                break;
            }
            case 0:
                std::cout << "Program Ended. Thank You!\n";
                break;
            default:
                std::cout << "Invalid choice! Try again.\n";
        }
    } while(choice != 0);

    return 0;
}
```

Output:

```
========= Supplementary Activity Menu =========
1. Swap Elements in a List
2. Convert Kelvin to Fahrenheit
3. Calculate Distance Between Two Points
4. Triangle Calculations (Perimeter, Area, Classification)
0. Exit
================================================
Enter your choice: 1
================================================
Enter how many numbers you want in the list: 4

Enter 4 numbers:
Number 0: 4
Number 1: 5
Number 2: 6
Number 3: 7

Before swapping: 4 5 6 7
Enter first position to swap (0-based index): 0
Enter second position to swap (0-based index): 3

After swapping: 7 5 6 4
```

```
========= Supplementary Activity Menu =========
1. Swap Elements in a List
2. Convert Kelvin to Fahrenheit
3. Calculate Distance Between Two Points
4. Triangle Calculations (Perimeter, Area, Classification)
0. Exit
================================================
Enter your choice: 1
================================================
Enter how many numbers you want in the list: 4

Enter 4 numbers:
Number 0: 4
Number 1: 5
Number 2: 6
Number 3: 7

Before swapping: 4 5 6 7
Enter first position to swap (0-based index): 0
Enter second position to swap (0-based index): -1
Invalid indices for swapping!

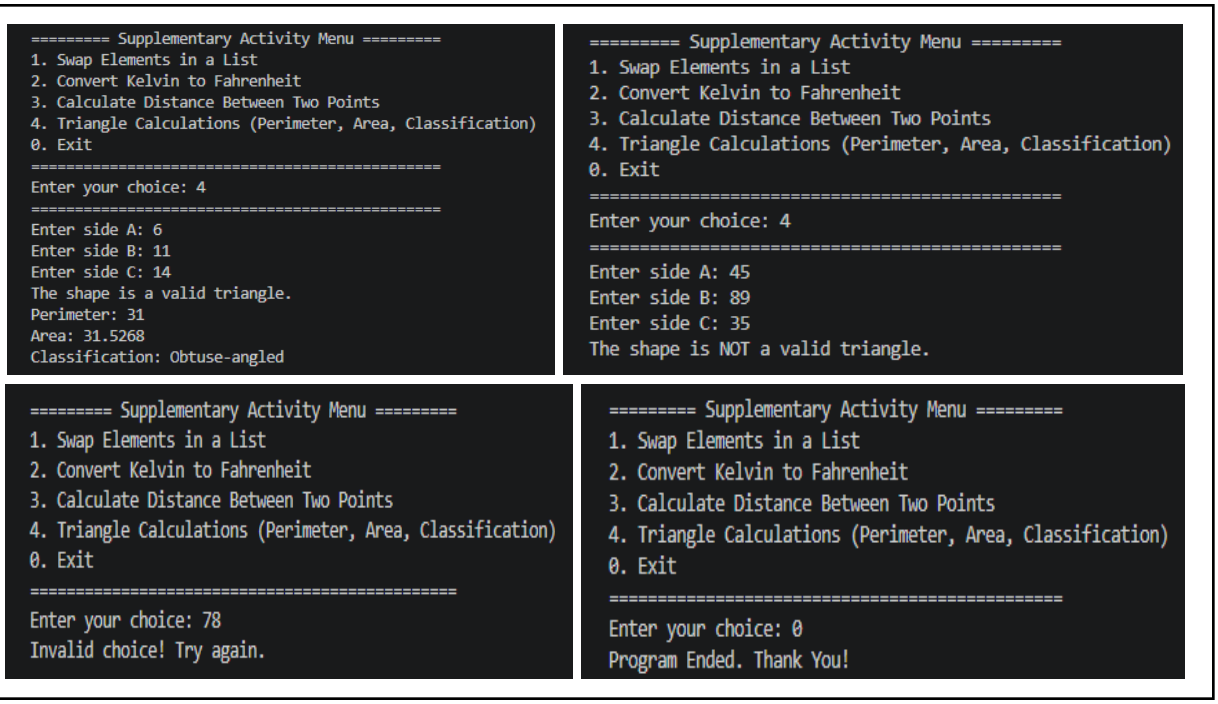
After swapping: 4 5 6 7
```

```
========= Supplementary Activity Menu =========
1. Swap Elements in a List
2. Convert Kelvin to Fahrenheit
3. Calculate Distance Between Two Points
4. Triangle Calculations (Perimeter, Area, Classification)
0. Exit
================================================
Enter your choice: 2
================================================
Enter temperature in Kelvin: 719.67
719.67 K = 835.736 F
```

```
========= Supplementary Activity Menu =========
1. Swap Elements in a List
2. Convert Kelvin to Fahrenheit
3. Calculate Distance Between Two Points
4. Triangle Calculations (Perimeter, Area, Classification)
0. Exit
================================================
Enter your choice: 3
================================================
Enter x1: 52.13
Enter y1: 49.56
Enter x2: 87.14
Enter y2: 74.28
The distance between (52.13, 49.56) and (87.14, 74.28) is: 42.8577
```

```
========= Supplementary Activity Menu =========
1. Swap Elements in a List
2. Convert Kelvin to Fahrenheit
3. Calculate Distance Between Two Points
4. Triangle Calculations (Perimeter, Area, Classification)
0. Exit
================================================
Enter your choice: 4
================================================
Enter side A: 45
Enter side B: 45
Enter side C: 15
The shape is a valid triangle.
Perimeter: 105
Area: 332.779
Classification: Acute-angled
```

```
========= Supplementary Activity Menu =========
1. Swap Elements in a List
2. Convert Kelvin to Fahrenheit
3. Calculate Distance Between Two Points
4. Triangle Calculations (Perimeter, Area, Classification)
0. Exit
================================================
Enter your choice: 4
================================================
Enter side A: 7
Enter side B: 24
Enter side C: 25
The shape is a valid triangle.
Perimeter: 56
Area: 84
Classification: Right-angled
```



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammar Correction for my Code Analysis**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.